

Mystery SQL

A fim de otimizar o trabalho em grupo dividiremos a realização do trabalho em 5 fases diferentes cada uma delas realizada por um grupo diferente, a saber:

1. Game Design e Lógica do Enredo
2. Modelagem do Banco de Dados (Schema)
3. Geração de Dados (População do Banco)
4. Desenvolvimento da Plataforma (Interface)
5. Playtesting e Balanceamento

Cada uma das mais importante a menor importante, dado que temos inúmeros trabalhos e coisas para estudar esta semana devemos focar mais nos passos 1, 2 e 3, desenvolver uma interface mínima e testar o "Caminho Feliz" do Misterio (As Queries que se realizadas corretamente solucionam o misterio)

Grupo 1: Let's create mystery and a relational scheme

- Integrante 1:
- Integrante 2:
- Integrante 3:
- Integrante 4:

Este grupo estará encarregado de criar de maneira conceitual um Misterio SQL inspirado no [SQL Murder Mystery](#). É interessante que o tema do misterio esteja relacionado com os trabalhos menores, dos integrantes do grupo temos 3 temas diferentes: Airbnb, E-class e Uber, se possível tentem integrar eles.

Algumas das tarefas a serem realizadas:

- Definir o Evento Principal: Qual é o crime ou mistério? (ex: um roubo de dados, um desaparecimento, um assassinato). Onde e quando ocorreu?
- Criar a Cadeia de Pistas: Desenhe o caminho lógico (o caminho feliz) que o jogador deve seguir. Por Exemplo:
 - Passo 1: Acessar o boletim de ocorrência na data X e local Y.
 - Passo 2: Descobrir o nome de uma testemunha.
 - Passo 3: Cruzar o depoimento da testemunha com a tabela de placas de carros.
 - Passo 4: Encontrar o suspeito final cruzando registros de catracas e histórico de compras.
 - Passo 5: ...

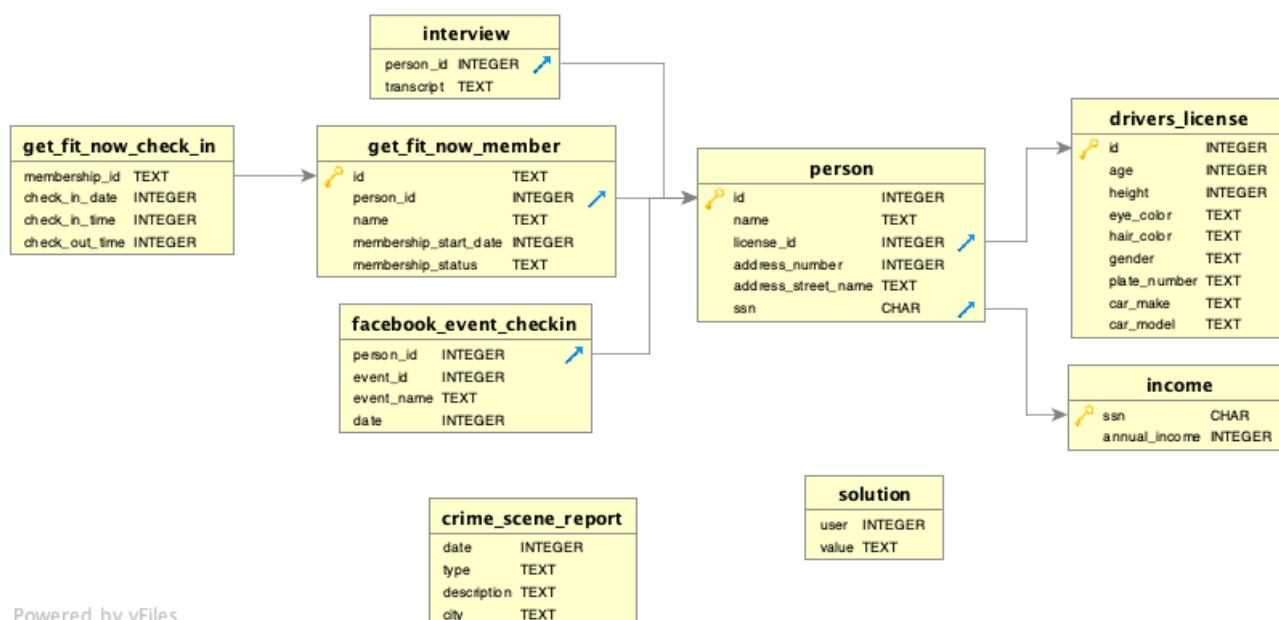
No SQL Murder Mystery estes passos não são explicitamente dados, unica informação que eles dão é a seguinte:

A crime has taken place and the detective needs your help. The detective gave you the crime scene report, but you somehow lost it. You vaguely remember that the crime was a murder that occurred sometime on Jan.15, 2018 and that it took place in SQL City. Start by retrieving the corresponding crime scene report from the police department's database.

E a partir desse passo 1 o jogador consegue inferir os proximos passos.

- Criar "Red Herrings" (Pistas Falsas): O banco de dados precisa ter outras histórias acontecendo paralelamente para que o jogador precise refinar seus JOINS e cláusulas WHERE.
- Modelar o banco de dados
 - o que queremos afinal é o modelo logico, podem fazer primeiro o modelo conceitual e depois o logico se for mais facil, ou fazer diretamente o logico
 - É interessante que esteja na 3FN, mas isso dependera de como o misterio seja modelado

Exemplo de como deveria ser a modelagem do banco de dados



Exemplo do que seria uma Red Herring

O Cenário

Imagine que o jogador encontrou o boletim de ocorrência de um roubo e extraiu a seguinte pista de uma testemunha:

"O ladrão fugiu em um carro. Eu não vi a placa toda, mas começava com **H42W**. Era um **homem**."

Passo 1: A Armadilha (O Red Herring)

O jogador vai até a tabela de carteiras de motorista (`carteira_motorista`) e faz uma busca usando o operador `LIKE` :

```
SELECT id, nome, placa, genero
FROM carteira_motorista
WHERE placa LIKE 'H42W%' AND genero = 'masculino';
```

Resultado da Query:

id	nome	placa	genero
8471	Roberto Dias	H42W99	masculino
9012	Carlos Silva	H42W12	masculino

O Problema: A query retornou duas pessoas. O jogador apressado pode achar que é o Roberto Dias e acusá-lo. Mas **Roberto é o Red Herring**. Ele é apenas um cidadão inocente que teve a infelicidade de ter uma placa parecida com a do ladrão.

Se o jogo for bem construído, acusar o Roberto resultará em "Você prendeu a pessoa errada!".

Passo 2: A Necessidade de Refinamento

Para resolver o empate, o jogador precisa de mais informações. Ele volta aos depoimentos ou aos relatórios policiais e encontra um detalhe adicional de outra testemunha:

"Eu vi o suspeito saindo da academia 'Maromba Fitness' no dia do crime (14 de Maio de 2026)."

Agora, o jogador descobre que precisa cruzar a tabela de motoristas com a tabela de acessos à academia (`catraca_academia`).

Passo 3: A Query Correta usando JOIN

Para eliminar a pista falsa, o jogador precisa fazer um `JOIN` entre as carteiras de motorista, os dados das pessoas e os registros da catraca, filtrando pela data específica:

```
SELECT c.nome, c.placa, a.data_acesso, a.nome_academia
FROM carteira_motorista c
```

```
JOIN pessoa p ON c.id = p.id_carteira
JOIN catraca_academia a ON p.id = a.id_pessoa
WHERE c.placa LIKE 'H42W%'
      AND c.genero = 'masculino'
      AND a.data_acesso = '2026-05-14'
      AND a.nome_academia = 'Maromba Fitness';
```

Resultado da Query:

nome	placa	data_acesso	nome_academia
Carlos Silva	H42W12	2026-05-14	Maromba Fitness

Como o Red Herring foi construído nos bastidores?

Quando o grupo encarregado de popular o banco de dados na Fase 3, terão que, por exemplo, fazer o seguinte:

1. Cria o Culpado (Carlos Silva): Você insere o Carlos com a placa H42W12 e garante que ele tenha um registro na tabela `catraca_academia` no dia 2026-05-14.
2. Cria a Pista Falsa (Roberto Dias): Você insere o Roberto com a placa H42W99. Para deixar a armadilha ainda melhor, você pode até colocar o Roberto como cliente da "Maromba Fitness", mas você registra a entrada dele na catraca no dia 2026-05-12 (dois dias antes do crime).

Dessa forma, o jogador é recompensado não apenas por saber escrever comandos SQL básicos, mas por entender a lógica relacional e por prestar atenção em todos os detalhes do "boletim de ocorrência" (`WHERE data_acesso = '...'`).

É importante que os Red Herring sejam especificados/explicados nesta parte da modelagem do jeito certo para que depois o grupo de criação dos dados não precise se preocupar com a lógica mas somente com a criação do banco de dados

Grupo 2: let's create data

- Integrante 1:
- Integrante 2:

Este é o passo mais trabalhoso. Um banco vazio ou com 10 linhas não oferece desafio. Você precisará de milhares de registros falsos (ruído) para esconder a agulha no palheiro.

Scripts de Mock Data: Ferramentas como a biblioteca Faker (em Python) são ideais para gerar rapidamente milhares de nomes, endereços e datas plausíveis. Como alternativa, você pode criar scripts personalizados para gerar arquivos CSV com controle preciso sobre a distribuição dos dados

controle preciso sobre a distribuição dos dados.

A "Injeção" da Verdade: Após gerar os dados aleatórios, você precisa inserir os dados cruciais (o assassino, a testemunha, os logs exatos) forçando-os a ter os IDs e chaves estrangeiras corretas para que o mistério faça sentido matemático.

Geração do Arquivo de Banco de Dados: Compile tudo em um único arquivo de banco de dados (o SQLite é a melhor escolha pela portabilidade). Então:

1. Criem o script de geração: Um arquivo gerar_dados.py que usa o sqlite3 e Faker para criar o arquivo .db e popular com os milhares de registros
2. Suba o banco: Deixem o arquivo .db disponível para o grupo 3 que irá implementar a interface.

Grupo 3: Let's make it easier for the player to access.

• Integrante 1:

Nesta parte não saberia como fazer exatamente, mas pensei em criar um repositório github com um docker file que instale automaticamente o SQLite quando criado um Codespace e deixar um read-me simples com a pista inicial e instruções básicas de como realizar as queries e visualizar as tabelas

Todos os grupos: let's have fun

Finalmente então todos do grupo testamos o mistério e verificamos que funciona de maneira plausível. seria interessante um de nós não saber as respostas do mistério e fazer ele quando finalizado